Muzz Chat

Android exercise: implementation decisions, assumptions, limitations and next steps · Pedro Loureiro · November 2026

I built a single chat screen with Jetpack Compose, where you chat with Sarah. A message you send appears at once, and a mock server then confirms it. A "Reply as Sarah" button sends the typed text as her, so you can try both sides on one device. Room stores the messages, so they survive restarts, and the list pages in older ones as you scroll up. The repository has more detail in the README, `CONTEXT.md` (glossary), `docs/architecture.md`, `docs/api.md`, `docs/decisions.md`, `docs/design-system.md` and four ADRs.

1. Approach
2. Architecture and system modelling
3. Tech stack
4. API assumptions
5. Mobile assumptions
6. Implementation decisions
7. Limitations
8. What I would do with more time

1 Approach

I built this as if it were an isolated feature that would really be integrated into Muzz, and I made it as production-ready as the exercise allowed. A chat screen that only appends rows to a local list would meet the brief. It would not survive contact with a real backend, though. Messages arrive late, twice, out of order or not at all. History is longer than memory. And neither the device's clock nor its sends can be trusted to order a conversation.

So most of my time went into three things. I designed the system around a server-authoritative message model, with a stable identity and a gap-free order. I built pagination and sync: cursor paging from Room, and a catch-up that finds and fills missed messages. And I wrote a mock backend that behaves like a real one, with latency, an echo on an incoming stream, idempotent sends, a server-assigned order and history pages. The app goes through the same paths a real network would use.

It is not plug and play. A real integration would bring its own API, authentication, error cases and product rules. I did write down the contract the app relies on (section 4), and the mock sits behind the same interfaces a real data source would implement. Integrating it should mostly come down to replacing the data source, with no redesign of the feature.

I did ask myself whether this was over-engineering for an exercise. I decided I could not ignore the mechanisms a real chat needs in order to work. Where I left something out, such as failure handling, edits or a chat list, I list it as a limitation, with the design for adding it. I also built a small design system from the Muzz press kit (palette, launcher icon, splash screen), so the result is on brand.

2 Architecture and system modelling

Layers and features

Each feature uses Clean Architecture. The layers are packages in one Gradle module, and dependencies point towards the domain. There are two features: `chat` is the screen and its messages, and `user` gives `chat` the current user. A feature may import only another feature's `domain`, and features depend one way (`chat → user`). The compiler does not check packages, so a unit test, `ArchitectureTest`, scans imports and fails when a rule is broken.

```
ui  ──>  domain  <──  data           di ──> ui, domain, data   (Koin)

chat/ui      ChatScreen, ChatViewModel, ChatItemMapper, stateless components
chat/domain  Message, ChatRepository, SendMessageUseCase, OtherUserDevice (mock)
chat/data    ChatRepositoryImpl, Room (messages, chat_sync), MessageRemoteMediator,
             ChatRemoteDataSource + MockChatRemoteDataSource, MockChatApi
user/domain  User, UserRepository              user/data  UserRepositoryImpl, mock remote
```

I add a use case only when an operation has logic of its own. `SendMessageUseCase` trims and validates the text and builds the message. Observing messages only forwards a call, so the ViewModel calls the repository directly. The repository coordinates two data sources. The remote one is the messaging service, with send, history pages and one incoming stream. The local one is Room, the single source of truth. `ChatRepositoryImpl` is the only place that writes messages.

Domain model

Concept	Model
User	A stable UUID `id` and a `name`. The current user comes from `UserRepository.currentUser()`, backed by a `me()` call.
Chat	A conversation, identified by a server `chatId`. Every message belongs to one Chat, and every query and remote call is limited to one `chatId`.
Message	A sealed class. **Pending** means this device sent it and the server has not stored it yet: it has the device time and no server id or order. **Confirmed** means the server has stored it: it has a `serverMessageId`, a `seq` and the server timestamp. Both carry `chatId`, `senderId`, `clientMessageId`, `Content` (type + value; only `TEXT`) and `timestampMillis` (epoch millis, UTC).
Identity	`(senderId, clientMessageId)`. The device generates a UUID per message, and the server returns it on every copy, from every sender. That pair is the Room primary key, the UI list key and the server's idempotency key. A message keeps one key from pending to confirmed, so its bubble never jumps.
Order	`seq`: a per-Chat number the server assigns, starting at 1 with no holes. Only `seq` orders confirmed messages, and pending ones come after them by device time. Timestamps are for display and grouping only. A missing `seq` means a missed message, and `seq` 1 is the start of the history.
Chat sync state	The app stores it per Chat, in the same transaction as the messages that move it: `oldestSeq` (where older history continues), `newestSeq` (newest `seq` with no gaps below it) and `reachedStart`.
ChatItem (UI)	A renderable row: a `Message` (direction, text, grouped, delivery status) or a `TimestampHeader`. `ChatItemMapper` builds it. The mapper is pure Kotlin, and I wrote it test-first.

Flows

```
Send     ViewModel → SendMessageUseCase (validate, build Pending) → Repository
         → insert pending row in Room (shown at once, faded with "sending" dots)
         → remote.sendMessage → response and echo on the stream carry the stored copy
         → first copy upserts by (senderId, clientMessageId): the row is confirmed in place
         → second copy writes the same values: no change

Receive  remote.incomingMessages() → same upsert, for the open Chat only
         → seq == newestSeq + 1 moves newestSeq on; a skipped seq is stored and starts a catch-up

History  Paging 3, Room as the PagingSource (20 rows a page, 40 on a first read),
         a RemoteMediator per Chat (60 messages a server request)
         REFRESH: no sync state → newest page; otherwise catch up after newestSeq until hasMore = false
         APPEND (scroll up): page before oldestSeq, until the server says hasMore = false
         Paged query returns each message with its next neighbour, for the bubble tail

Render   PagingData.map → ChatItem.Message;  insertSeparators → TimestampHeader,
         then StartOfChat above the oldest header once the whole history has loaded
         LazyColumn(reverseLayout = true), keyed by the message identity
```

Mock boundary

I marked everything that replaces something the exercise does not have with `MOCK DATA` and kept it isolated, so it can be deleted without touching the layers above:

- `MockChatApi` plays the server. It holds a 200-message seed from `assets/`, plus the messages sent to it. It keeps them in a file, so it never reuses a `seq` after a restart. `MockChatRemoteDataSource` adds ~250ms of latency and a `SharedFlow` in place of the WebSocket.
- `MockUserRemoteDataSource` answers `me()` with a fixed user.
- The chat id and Sarah's name and user id are constants beside `MainActivity`. They reach the screen as parameters, where navigation arguments would come from.
- "Reply as Sarah" goes through `OtherUserDevice`, a mock contract that sends as Sarah through the mock server. Her message comes back on the incoming stream like one from any other device, so this device never writes it directly.

3 Tech stack

Choice	Why	Trade-off
Kotlin 2.3, coroutines and Flow	My default on Android. Room, Paging and Compose all speak `Flow`, so the UI observes the database end to end with no adapters.	None for this scope.
Jetpack Compose (Material 3 as the base)	Declarative, stateless components that are quick to build and preview. The components are custom and read Muzz tokens. Material provides only `Scaffold` and theming.	Custom components take more code than stock Material ones. In return they match the brand and keep every state explicit.
Room 2.8 (with KSP)	The Jetpack database, with SQL checked at compile time, `Flow` queries, transactions for message and sync-state writes, and a ready `PagingSource`.	The generated paging source uses `LIMIT/OFFSET`. It also drops a change made during its first load, so I needed a small wrapper.
Paging 3.5 with `RemoteMediator`	Jetpack's library for paging from a local database while fetching from the network. It has placeholders, prefetch, and `map` / `insertSeparators` for the display rules.	Hard to learn. Transformations see one row at a time, so the query has to supply the next neighbour. `paging-common` reaches the domain layer (an accepted deviation).
Koin 4.2	Lightweight DI with a Kotlin DSL. It needs no annotation processing, builds fast, takes little setup, and supports Compose and ViewModels with parameters (`chatId`, Sarah's id).	Koin resolves the dependency graph at run time, so a missing binding fails when the app runs. Hilt/Dagger would catch it at compile time. For a large app, Hilt's compile-time checks would be worth its setup.
kotlinx.serialization	Kotlin-first, and free of reflection through a compiler plugin. I use it for the mock seed, the mock server's file and the DTOs, as a real API client would.	A different library from Moshi or Gson, if the main app uses one of those.
`java.time`, minSdk 26	Correct calendar-day and time-zone arithmetic for headers, with no desugaring and no `SimpleDateFormat`.	The app does not support Android 7.x (API 24 and 25).
JUnit 4, Robolectric, paging-testing	Every test runs on the JVM, Room included, against a real SQLite in memory, so `./gradlew test` needs no emulator. `asSnapshot()` tests the paging wiring.	Robolectric's SQLite is not the one on a device, and there are no instrumented or UI tests.
One Gradle module, layers as packages	Fast to set up and build for two small features. `ArchitectureTest` enforces the import rules.	The compiler does not enforce boundaries, and build caching is per module. In a larger codebase, splitting into modules is the first step.
No navigation library	The app has one screen. The chat's arguments arrive where navigation arguments would come from.	A chat list would need Navigation Compose.
`core-splashscreen`	The same branded splash on API 26 to 30 as on 31+.	None.

4 API assumptions

I wrote the app against a backend contract that the exercise does not have. The client relies on every guarantee below. A real backend must provide each one, or the client must change.

Area	Assumption
Authentication	Every call and the incoming stream carry the signed-in user's token. The server knows the caller from the token alone.
Sender	The server sets `senderId` from the token. An outgoing message has no sender field, so a device can send only as its own user.
Current user	`me()` returns the signed-in user, with the same id the server puts in `senderId`. The client compares the two to tell outgoing from incoming messages.
Send	`sendMessage(chatId, {clientMessageId, content, timestampMillis})` returns the stored message. The server assigns a random UUID `serverMessageId`, the Chat's next `seq` and its own timestamp.
Idempotency	`(senderId, clientMessageId)` is unique on the server. A resend returns the message already stored and stores nothing new, which is what makes retries safe.
Ordering	Each Chat numbers its messages by `seq`, from 1 with no holes. The server assigns it in the same transaction that stores the message.
Immutability	Stored messages never change: no edits or deletes. So a late or repeated copy cannot overwrite newer state.
Incoming stream	One stream (a WebSocket) for every Chat the user belongs to. It includes the sender's own messages, as the echo that confirms them. Every copy carries `clientMessageId`. Delivery order does not matter, and the stream may miss messages.
History	`fetchMessagesBefore(chatId, beforeSeq?, limit)` returns older messages (the newest page when there is no cursor). `fetchMessagesAfter(chatId, afterSeq, limit)` returns newer ones. Each returns `{messages, nextCursor, hasMore}`. On an older page, `hasMore = false` means the start of the Chat.
Time	Timestamps are epoch milliseconds, so always UTC. The app uses them for display and grouping only.
Left out	Network errors, timeouts, 4xx/5xx, rate limits, payload limits, reconnection, token refresh and sign-out. The mock is always reachable and always succeeds.

5 Mobile assumptions

The chat screen expects these things from the rest of a real Muzz client, which this exercise does not include:

Area	Assumption
Navigation into a Chat	The screen that opens a Chat, such as a chat list or a match, passes the Chat's `chatId`, the other user's id and their display name as navigation arguments. The chat screen shows the name in the top bar at once, with no extra request. Here `MainActivity` passes mock constants in their place.
Session	A user is already signed in, with a valid token, before the chat opens. Sign-in, token refresh and sign-out belong to another part of the app. The chat reads the current user from `UserRepository` and does not cache it.

One open Chat	The app stores incoming messages for the Chat that is open, while its ViewModel is alive. It fetches other Chats' messages through the catch-up when they are opened. Once a chat list exists, an app-level sync would take this over.
Device clock	The app uses the device time only for a pending message, until the server's time replaces it. A wrong clock cannot reorder the conversation.
Time zone and locale	Header labels use the device's time zone and locale, which the app reads each time the list changes.
Local data	Room is a cache of the server's history, and the app can rebuild it from the server. The app has never been released, so schema changes need no migrations yet.
Platform	Android 8.0+ (minSdk 26, for `java.time` without desugaring), phones in portrait and landscape, English, light theme.
Configuration changes and process death	The typed text lives in the ViewModel, so it survives rotation. Messages live in Room, so they survive the app being killed. A send in flight is tied to the ViewModel (see Limitations).

6 Implementation decisions

The brief's rules

Rule	How it is applied
Newest at the bottom; sent right, received left, in bubbles	A `LazyColumn` with `reverseLayout`. The mapper sets the direction by comparing `senderId` with the current user's id, never from the code path that stored the message.
Small spacing: same sender and < 20s apart	The message is *grouped*. It gets 2dp below it instead of 8dp and no bubble tail, so the tail marks the last bubble of a run. Paging maps one row at a time, so the paged query supplies the next message.
Section header: previous message > 1h older, or none	`insertSeparators` adds the headers. The label depends on the message's age: `11:59` today, `Thursday 11:59` within 6 days, and `12 Mar 2026 11:59` before that. A bare weekday would be ambiguous for older history.
Trigger the other user's messages	A "Reply as Sarah" button beside Send. It sends through the mock server, as described in Mock boundary.
Bonus: persistent DB observed by the UI	Room is the only source of truth. The UI observes it through `Flow<PagingData>`, and nothing else holds a copy of the messages.

How message identity and order evolved

Message identity was the hardest part to get right. It changed three times as I tested each version against what a real server and a second device would do:

Version	Design	Why it changed
1. Device id, status flip	The device generated the message id and timestamp. The echo flipped the stored message from `SENDING` to `SENT`.	The device owned identity and order, so a wrong clock or a second device could reorder the conversation. A real server owns both.
2. Server UUIDv7	The server generated a time-ordered UUIDv7 that both ordered and paged messages. The app stored a pending message under a device `localId`, and the echo replaced it with a delete and an insert in one transaction.	UUIDv7 values are not consecutive, so the app could not tell whether a message between two ids was missing. Same-millisecond ties were arbitrary. Messages from other devices had no `localId`, so the UI key changed source, and confirming took two writes.
3. Per-Chat `seq`	A server-assigned sequence with no holes orders and pages each Chat. A sync state per Chat and a forward catch-up fill gaps exactly.	That solved ordering. The pending/confirmed swap and the inconsistent UI key remained.
4. `(senderId, clientMessageId)` + `seq` (current)	The server returns the sender's `clientMessageId` on every message. That pair is the key everywhere, and the first server copy confirms the pending row in place with one upsert. `serverMessageId` is an opaque id and nothing more, and `seq` only gives the order.	It has not changed since.

The final design gives every message one key from the moment it is sent, on every device. Confirming is one idempotent write, retries are safe, and the app detects a missing message exactly. It has costs too:

- The server must assign `seq` atomically per Chat, which serialises writes within a Chat. One-to-one chats can live with that. Busy group chats would need batching or sharding.
- A deleted message must keep its `seq`.
- The client keeps more state, a sync row per Chat.
- Each revision cost me time in docs, code and tests. Getting identity right early made paging, echoes and the scrolling behaviour straightforward afterwards.

Main decisions

Decision	Why
Optimistic send, confirmed in place by the first server copy	Feedback is instant, each confirmation is one write, and the bubble never jumps. By construction, a repeated echo does no harm. (ADR-0001)
Server-assigned per-Chat `seq`, with no holes	The app detects missed messages exactly and knows where the history starts. A time-ordered UUID or a timestamp can only guess at gaps. (ADR-0003)
Paging 3 with Room as the source and a `RemoteMediator`	It handles history of any length. The mediator fetches by cursor, so new messages never cause repeated or skipped rows. On open, a forward catch-up fills gaps without dropping stored history. (ADR-0003)
Clean Architecture as packages, with a test enforcing imports	Clear boundaries, without the build setup of separate modules for two small features. (ADR-0002, ADR-0004)
Use cases only where there is logic	I avoided use cases that only forward a call. To observe messages, the ViewModel calls the repository.
A separate `user` feature	It removes the global "me" constant. `chat` asks `UserRepository` for the current user, as it would with a real session. (ADR-0004)
Stateless composables	The screen owns all state, including scroll, press and focus. Components render only what they receive.
Two-tier design tokens, Muzz palette	Components read semantic roles and never raw values, so dark mode becomes a palette change. The launcher icon and splash screen are branded.
Deliberate visual departures from the brief	The input is filled instead of outlined. Sending shows pulsing dots instead of ticks, because the app knows only "sending" and "sent". There is no overflow menu, because I had no action to put in it.
Scrolling	New messages scroll into view only when you are at the bottom. Nothing interrupts you while you read older history: Jump to latest appears instead. Your own sends snap to the newest message, so what you send always comes into view. Reply as Sarah simulates a received message, so it does not snap.
Loading states	On a fresh install, a centred spinner shows until the first messages arrive. While an older page comes from the server, a small spinner on a bordered surface floats at the top of the list. It is an overlay outside the list rows, so the rows never shift. Once the whole history has loaded, a "Start of conversation" pill marks the oldest message, so the user knows there is nothing older.
Server pages larger than Room's first read	Older history used to stop loading after a page or two. Paging 3.5.3 drops an APPEND requested while the mediator's APPEND still runs. Storing a page starts a new generation. With 20-message server pages, that generation's first Room read (40 rows centred on the user) always reached the end of stored history and requested an APPEND at exactly that moment. Server requests now fetch 60 messages, more than half that read. The request then comes later, from a read that scrolling triggers, when the mediator is idle. A test guards the ratio.

Room paging workaround	Room 2.8.5 drops a change made to the table during the paging source's first load, which left a confirmed message shown as sending. `FirstLoadSafePagingSource` re-invalidates when that happens.
Tests	I wrote the logic test-first: the mapper, use case, ViewModel, repository, mediator, mappers, converters and date utils. The DAO and paged query run on an in-memory Room database with Robolectric. I did not test composables.

7 Limitations

- The app has no failure paths. The mock never fails, so a failed send would stay "sending" with no retry, and the UI has no error or connection state.
- Sends are tied to the screen. If the ViewModel is cleared before the server answers, the message stays pending for good, and nothing resends it.
- Outgoing status is only "sending" or "sent". There are no delivered or seen states and no sent or seen times.
- Messages are text only, with no edit, delete, reply, or clearing the chat. To start over, you clear the app's data.
- There is one Chat and no chat list. Incoming messages are stored only while the chat is open, and there are no notifications.
- While you read history, the only sign of a new message is Jump to latest, which shows whenever the newest message is out of view.
- The page-size ratio works around the Paging race and leaves the race in place. If someone changes the sizes without keeping the ratio, the race comes back, and a test catches that.
- Header labels update only when messages change. An `11:59` label shown across midnight stays until the next message arrives.
- There is no memory or storage cap. Pages scrolled past stay in memory, and Room keeps every page the app fetched.
- The app does not cache the current user. With a real backend, every send and every list open would wait for `me()`.
- The app has a light theme and English only. White on Muzz pink is about 3.6:1, below WCAG AA for body text. I kept it for the brand.
- There is no Room schema history and there are no migrations. The app was never released, so the schema changes in place.
- Layers are packages in one module. A test checks the boundaries, and the compiler does not.

8 What I would do with more time

Product features

Feature	Approach
Sent at and seen at times	Show the server time of each outgoing message, on the last bubble of a run or on tap. Show the seen time from the other user's read receipt.
Delivered and seen states	The server keeps a `deliveredSeq` and `readSeq` per member of a Chat, and sends a receipt event when one moves. `seq` has no holes, so one number per Chat marks every message up to it, and no per-message receipts are needed. A message's status follows from `seq ≤ readSeq`, and the bubble renders it as ticks.
Network errors and retries	Pending rows become an outbox. A WorkManager job with a network constraint and exponential backoff sends them, so a send outlives the screen. Network errors and 5xx are retried with the same `clientMessageId`, which server idempotency makes safe. A 4xx response, or running out of retries, marks the message `FAILED` with tap to retry.
Delete messages	A tombstone: the message keeps its `seq` and loses its content, with a `deleted` flag, so it never becomes a hole. The stream delivers it as its own event.
Edit messages	`editedAt` plus a server `changeSeq` that rises with every change. The app applies a change only when it is newer than the stored one, so a late event cannot overwrite a newer edit. The bubble shows an "edited" label.
Simple account information for Sarah	Tapping the top bar opens a profile sheet with her photo, name, age, location and a short bio, from a `user(id)` call in the `user` feature. The avatar shows her photo instead of an initial.
Reply to a specific message	A message carries the identity of the message it replies to, and the server returns a small preview. The bubble shows a quote. Tapping it scrolls to the original by `seq` and fetches its page if needed. You swipe a message to reply.
Dark mode	Components already read semantic roles only. Dark mode means a dark mapping from tier-2 tokens to primitives, following the system setting, and then a contrast check.
Images, documents, videos	New `ContentType` values with attachment metadata (URL, MIME type, size, dimensions, thumbnail). A WorkManager job uploads the file to media storage first, through a pre-signed URL, and then the app sends the message. Pending bubbles show upload progress, and Coil loads images.
Audio messages	Hold to record, with a cancel gesture. The recording uploads like other media, with a duration and waveform in the metadata, and plays inline with Media3.
"New message" pill while reading history	Count the messages that arrive while the newest message is out of view, and show a "New message" pill above the input, with the count when there is more than one. Tapping it jumps to the newest message and clears the count.

Engineering follow-ups

`docs/decisions.md` already has the steps for each one.

- **Chat list and app-level sync.** One handler for the stream that follows the app's foreground and stores every Chat. A `chats` table with the last message, activity time and unread count, a catch-up after every reconnect, and push notifications.
- **Error and connection state.** Render the paged list's errors with a retry, and add connection state to `ChatUiState`.
- **Room migrations.** Export the schema, version every change, and test each migration with `MigrationTestHelper`.

- **Localisation.** Add translations and locale-aware date patterns, and check layouts with longer text and in right-to-left languages.
- **Gradle modules.** Split each feature into domain, data and presentation modules, so the compiler enforces boundaries.
- **Keep header labels current.** Invalidate the list at midnight and on time zone or locale changes.
- **Long histories.** Cap pages in memory, trim old messages from Room, and replace the generated paging source with one keyed by `seq`.
- **Cache the current user.** Keep the signed-in user on disk from sign-in to sign-out.